

PROGRAMMING II

FUNCTIONAL INTERFACES AND STREAMS

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026

FUNCTIONAL INTERFACES



FUNCTIONAL INTERFACE

A **functional** interface contains a single method *// aka Single Abstract Method*

- ▶ Like `java.lang.Comparator`

The semantics is purely functional

- ▶ The outcome of the method is based on its arguments only

Predefined functional interfaces are in the `java.util.function` package

- ▶ Primitive and generics versions
- ▶ It is good to annotate them with `@FunctionalInterface`

PREDICATE

```
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
}
```

A function taking an argument of type `T` and returning a `boolean`

- Often used to filter elements in a collection

```
Predicate<Integer> isEven = new Predicate<>() { // with an          1
    @Override                                // anonymous class    2
    public boolean test(Integer n) { return n \% 2 == 0; }          3
};                                           4
```

PREDICATE

```
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
}
```

A function taking an argument of type `T` and returning a `boolean`

- Often used to filter elements in a collection

```
Predicate<Integer> isEven = n -> n \% 2 == 0; // with a lambda      1
                                                                    2
System.out.println(isEven.test(2)); // prints 'true'                3
System.out.println(isEven.test(3)); // prints 'false'               4
```

SPECIALIZED PREDICATE

```
@FunctionalInterface
public interface IntPredicate {
    boolean test(int v);
}
```

Specialized versions for primitive types are also provided

- ▶ Same usage of the generics version

```
IntPredicate isEven = n -> n \% 2 == 0;

System.out.println(isEven.test(2)); // prints 'true'
System.out.println(isEven.test(3)); // prints 'false'
```

1
2
3
4

FUNCTION

```
@FunctionalInterface
public interface Function<T, R> {
    R apply(T t);
}
```

A function taking an argument of type `T` and returning a result of type `R`

- Often used to map one value to another

```
Function<String, Integer> strLen = str -> str.length();           1
                                                                    2
System.out.println(strLen.apply("hello")); // prints '5'         3
```

BiPREDICATE

```
@FunctionalInterface
public interface BiPredicate<T, U> {
    boolean test(T t, U u);
}
```

A function taking two arguments of types `T` and `U` and returning a `boolean`

- Often used to test conditions on two values

```
BiPredicate<Integer, Integer> isGreaterThan = (a, b) -> a > b;           1
                                                                           2
System.out.println(isGreaterThan.test(2, 5)); // prints 'false'          3
System.out.println(isGreaterThan.test(3, 1)); // prints 'true'           4
```


BiFunction

```
@FunctionalInterface
public interface BiFunction<T, U, R> {
    R apply(T t, U u);
}
```

A function taking two arguments of types **T** and **U** and returning a result of type **R**

- Often used to implement operations on two values

```
Function<Integer, Integer, Integer> sumInt = (a, b) -> a + b;
System.out.println(sumInt.apply(3, 9)); // prints '12'
```

1
2
3

CONSUMER

```
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

A function taking an argument of types `T` and returning no result

- Often used to model side-effects

```
Consumer<String> printUC = str -> System.out.println(str.toUpperCase()); 1
                                                                    2
printUC.accept("string"); // prints 'STRING'                             3
```

SUPPLIER

```
@FunctionalInterface
public interface Supplier<T> {
    T get();
}
```

A function taking no arguments and returning a value of type T

- Often used for value generation

```
Supplier<Double> randomNum = () -> Math.random();           1
                                                                    2
System.out.println(randomNum.get()); // prints a random decimal number 3
```

OTHER FUNCTIONAL INTERFACES

```
public interface UnaryOperator<T> extends Function<T, T>
```

- ▶ A function taking one argument of type `T` and returning a result of type `T`

```
public interface BinaryOperator<T> extends BiFunction<T, T, T>
```

- ▶ A function taking two arguments of type `T` and returning a result of type `T`

```
@FunctionalInterface
public interface Validator<T> { boolean validate(T input); }

Validator<String> noHtmlValidator = input -> {
    if (input.contains('<') || input.contains('>')) return false;
    return true;
};
```

METHOD REFERENCE

Compact representation of an functional interface invocation

`System.out::println` equivalent to `str -> System.out.println(str)`

static method	<code>Class::staticMethod</code>
instance method (params)	<code>object::instanceMethod</code>
non-static method (no param)	<code>Class::nonStaticMethod</code>
constructor	<code>Class::new</code>

METHOD REFERENCE (CONT'D)

Instance method of a specific object

```
String hexDigits = "0123456789ABCDEF";           1
IntFunction<Character> toHex = hexDigits::charAt;  2
// in place of i -> hexDigits.charAt(i)           3
                                                    4
System.out.println(toHex.apply(15)); // prints 'F' 5
```

Static method

```
Supplier<Double> randomNum = Math::random;         1
// in place of () -> Math.random()                 2
                                                    3
System.out.println(randomNum.get()); // prints a random decimal number 4
```

METHOD REFERENCE (CONT'D)

Instance method reference

```
Function<String, Integer> strLen = String::length;           1
// in place of str -> str.length()                          2
                                                                3
String[] arrStr = {"two", "four"}; // prints '3' and '4'     4
for (String str : arrStr) System.out.println(strLen.apply(str)); 5
```

Constructor

```
Function<String, Person> personBuilder = Person::new;        1
// in place of str -> new Person(str)                        2
                                                                3
Person sam = personBuilder.apply("Sam")                       4
System.out.println(sam); // prints 'Sam'                     5
```

STREAMS



STREAM API

The `java.util.stream` package (Java 8) contains classes for processing **streams**

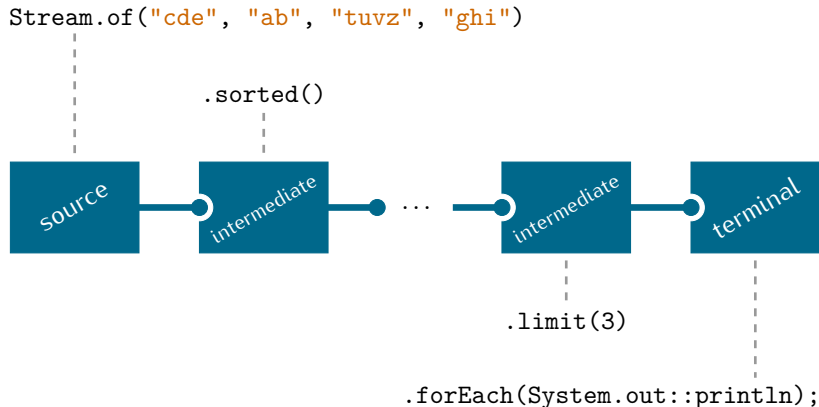
A `Stream<T>` is a sequence of **T** elements from a source

- ▶ Such a sequence supports data processing operations
- ▶ Which are defined by means of functional interfaces

Streams natively provide

- ▶ Pipelining
- ▶ Internal iteration
- ▶ Parallelization

STREAM API (CONT'D)



SOURCES

Stream from a variable list of arguments, with `Stream.of(T... elems)`

```
Stream<String> stream = Stream.of("a", "b", "c");
```

1

Stream from an array, with `Arrays.stream(T[] arr)`

```
String[] arr = new String[]{"a", "b", "c"};  
Stream<String> stream = Arrays.stream(arr);
```

1

2

Stream from a collection, with `Collection.stream()`

```
Set<String> set = Set.of("a", "b", "c");  
Stream<String> stream = set.stream();
```

1

2

UNWRAP STREAMS

List from a stream, with `Stream.toList()`

```
List<Integer> list = Stream.of(1, 2, 2, 3).toList();      1
// list contains [1, 2, 2, 3]                             2
```

Set from a stream, with `Stream.collect(Collector<T, A, R> collector)`

```
Set<Integer> set = Stream.of(1, 2, 2, 3).collect(Collectors.toSet());  1
// set contains [1, 2, 3]                                             2
```

Array from a stream, with `Stream.toArray(IntFunction<T[]> generator)`

```
Integer[] arr = Stream.of(1, 2, 2, 3).toArray(Integer[]::new);      1
// arr contains [1, 2, 2, 3]                                         2
```

STREAM GENERATION

Operation that repeatedly applies a unary operator on a seed to create a stream

```
Stream<T> iterate(T seed, UnaryOperator<T> generator)
```

Recall that a `UnaryOperator<T>` takes a `T` value and yields a `T` value

- ▶ Generate a **potentially infinite** number of values
- ▶ A new stream with the elements generated is returned

STREAM GENERATION (CONT'D)

Infinite generation

// lazy evaluation

```
Stream<Double> stream = Stream.iterate(Math.random(), d -> Math.random());  
// generate an unbounded sequence of random decimal numbers  
  
// equivalent, expecting a Supplier<T>  
Stream<Double> stream = Stream.generate(Math::random); // the computation halts (lazy evaluation)  
List<Double> list = stream.toList(); // the computation will never halt
```

Controlled generation

```
Stream<Integer> stream = Stream.iterate(0, i -> i + 1).limit(10);  
List<Integer> list = stream.toList();  
// generate the first ten natural numbers [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

INTERMEDIATE AND TERMINAL OPERATIONS

Intermediate operations return another `Stream<T>` object, as the source

- ▶ They do not modify the stream source
- ▶ They can be chained, by using the dot-notation

```
String[] arr = {"a", "c", "b", "c"};           1
Stream<String> distinct = Arrays.stream(arr).distinct().sorted(); 2
long count = Arrays.stream(arr).distinct().count(); 3
```

Terminal operations return a result, of a specific type

- ▶ Only a single terminal operation is allowed, if any

INTERMEDIATE AND TERMINAL OPERATIONS (CONT'D)

Intermediate operations return another `Stream<T>` object, as the source

- ▶ They do not modify the stream source
- ▶ But streams can only be consumed once

```
String[] arr = {"a", "c", "b", "c"};           1
Stream<String> stream = Arrays.stream(arr);    2
Stream<String> distinct = stream.distinct().sorted(); 3
long count = stream.distinct().count();       4
```

This will yield a runtime exception!

FILTER

Intermediate operation that applies a predicate on each stream element

```
Stream<T> filter(Predicate<T> predicate)
```

Recall that a `Predicate<T>` represents a boolean check on `T` values

- ▶ The predicate is checked on all elements of the stream
- ▶ A new stream with the elements satisfying the predicate is returned

FILTER (CONT'D)

With a lambda

```
String[] arr = {"abc", "cde", "ecd", "cba"};           1
Stream<String> stream = Arrays.stream(arr).filter(s -> s.contains("d")); 2
// stream contains "cde" and "ecd"                      3
```

With a method reference

```
Character[] arr = {'a', '3', '7', 'd'};                1
Stream<Character> stream = Arrays.stream(arr).filter(Character::isDigit); 2
// stream contains '3' and '7'                          3
```

INTERMEDIATE FILTERING

Discard duplicated elements: `Stream<T> distinct()`

Return the first `n` elements: `Stream<T> limit(int n)`

Discard the first `n` elements: `Stream<T> skip(int n)`

Sort elements according to **natural order**: `Stream<T> sorted()`

- ▶ Here, `T` must implement `Comparable`
- ▶ Or according to a comparator: `Stream<T> sorted(Comparator<T> comp)`

MAP

Intermediate operation that applies a transformation on stream elements

```
Stream<R> map(Function<T,R> mapper)
```

Recall that a `Function<T,R>` transforms `T` values into `R` values

- ▶ The transformation is applied on all elements of the stream
- ▶ A new stream with the transformed elements is returned

MAP (CONT'D)

With a lambda

```
String[] arr = {"abc", "cdef", "ec", "cba"};           1
Stream<Integer> stream = Arrays.stream(arr).map(s -> s.length()); 2
// stream contains 3, 4, 2 and 3                        3
```

With a method reference

```
String[] arr = {"aBc", "cd7f", "eC", "2Ba"};           1
Stream<String> stream = Arrays.stream(arr).map(String::toUpperCase); 2
// stream contains "ABC", "CD7F", "EC" and "2BA"      3
```

PREDICATE MATCHING

Terminal operations that apply a predicate on the whole stream

At least one element of the stream matches the predicate

```
boolean anyMatch(Predicate<T> predicate)
```

All elements of the stream match the predicate

```
boolean allMatch(Predicate<T> predicate)
```

No element of the stream matches the predicate

```
boolean noneMatch(Predicate<T> predicate)
```

PREDICATE MATCHING (CONT'D)

Find the first element of the stream (possibly matching a predicate)

```
Optional<T> findFirst()
```

Find an element of the stream (possibly matching the predicate)

```
Optional<T> findAny()
```

An empty optional is returned if the stream is empty

- For instance, when a previous filtering gives no elements

OPTIONAL

An **optional** represents a potential value

```
Integer[] arr = {-1, 4, -5, 0, 3}; 1
Optional<Integer> opt = Arrays.stream(arr).filter(e -> e <= 0).findAny(); 2
3
// check if an element has been found 4
if (opt.isPresent()) System.out.println(opt.get()); 5
else System.out.println(0); 6
7
// more compact, equivalent version 8
System.out.println(opt.orElse(0)); 9
```


STREAM ITERATION

Terminal operation that iterates over the stream elements

- ▶ The action is performed on all elements of the stream
- ▶ The stream is consumed

// cannot be reused

```
void forEach(Consumer<T> action)
```

```
// print a stream to screen 1
Integer[] arr = {-1, 4, -5, 0, 3}; 2
Arrays.stream(arr).filter(e -> e <= 0).forEach(System.out::println); 3
4
// count the elements of a stream 5
System.out.println(Arrays.stream(arr).filter(e -> e <= 0).count()); 6
```

FLAT MAP

Intermediate operation that extracts a stream from each stream element

```
Stream<R> flatMap(Function<T,Stream<R>> mapper)
```

Here `Function<T,Stream<R>>` transforms `T` values into `R` streams

- ▶ The transformation is applied on all elements of the stream
- ▶ A new stream `concatenating` all the resulting streams is returned
- ▶ Typically `T` represents multiple values (e.g., a collection)

FLAT MAP (CONT'D)

For arrays

```
Byte[][] arr = { { 1, 2, 3}, {4, 5, 6}, {7, 8, 9} };           1
Stream<Byte> stream = Arrays.stream(arr).flatMap(r -> Arrays.stream(r)); 2
// stream contains 1, 2, 3, 4, 5, 6, 7, 8 and 9                 3
```

For collections

```
List<List<Integer>> list = new LinkedList<>();                  1
list.add(new LinkedList<>()); list.add(new LinkedList<>());    2
list.get(0).add(1); list.get(0).add(3);                        3
list.get(1).add(7); list.get(1).add(9);                        4
Stream<Integer> stream = list.stream().flatMap(Collection::stream); 5
// stream contains 1, 3, 7 and 9                                 6
```

FLAT MAP (CONT'D)

Chaining again

```
Integer[][] arr = { { 1, -2, 3}, {4, -5, -6}, {-7, -8, 9} };      1
Arrays.stream(arr)                                              2
    .flatMap(Arrays::stream)                                    3
    .filter(i -> i > 0)                                         4
    .map(i -> String.format("%d is positive", i))              5
    .forEach(System.out::println);                             6
//                                                             7
// output:                                                      8
// '1 is positive'                                             9
// '3 is positive'                                           10
// '4 is positive'                                           11
// '9 is positive'                                           12
```



Q + A

